Práctica 2 — Despliegue en Tomcat

Alumno	**Alejandro Martínez Corral**
Módulo	Despliegue de Aplicaciones Web
Unidad	UD1 Arquitecturas Web

Paso 1. Levantar Tomcat

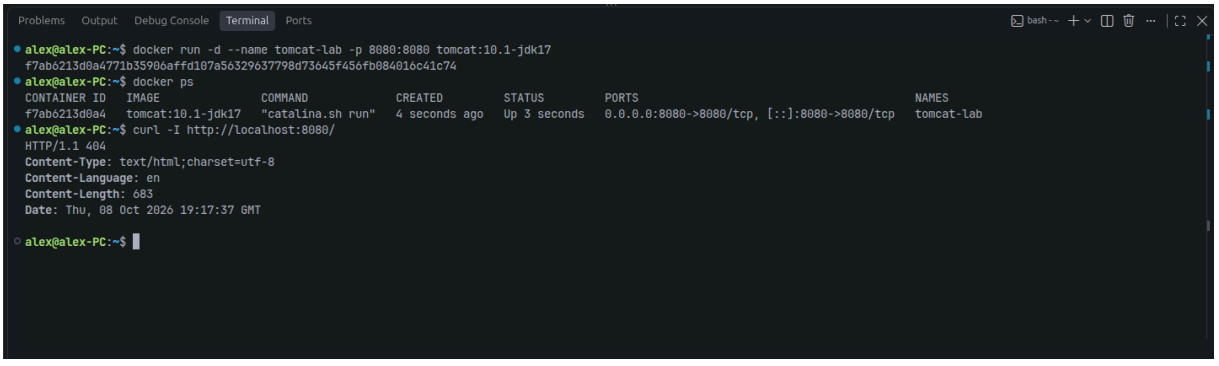
```
docker run -d --name tomcat-lab -p 8080:8080 tomcat:10.1-jdk17
docker ps
curl -I http://localhost:8080/
```

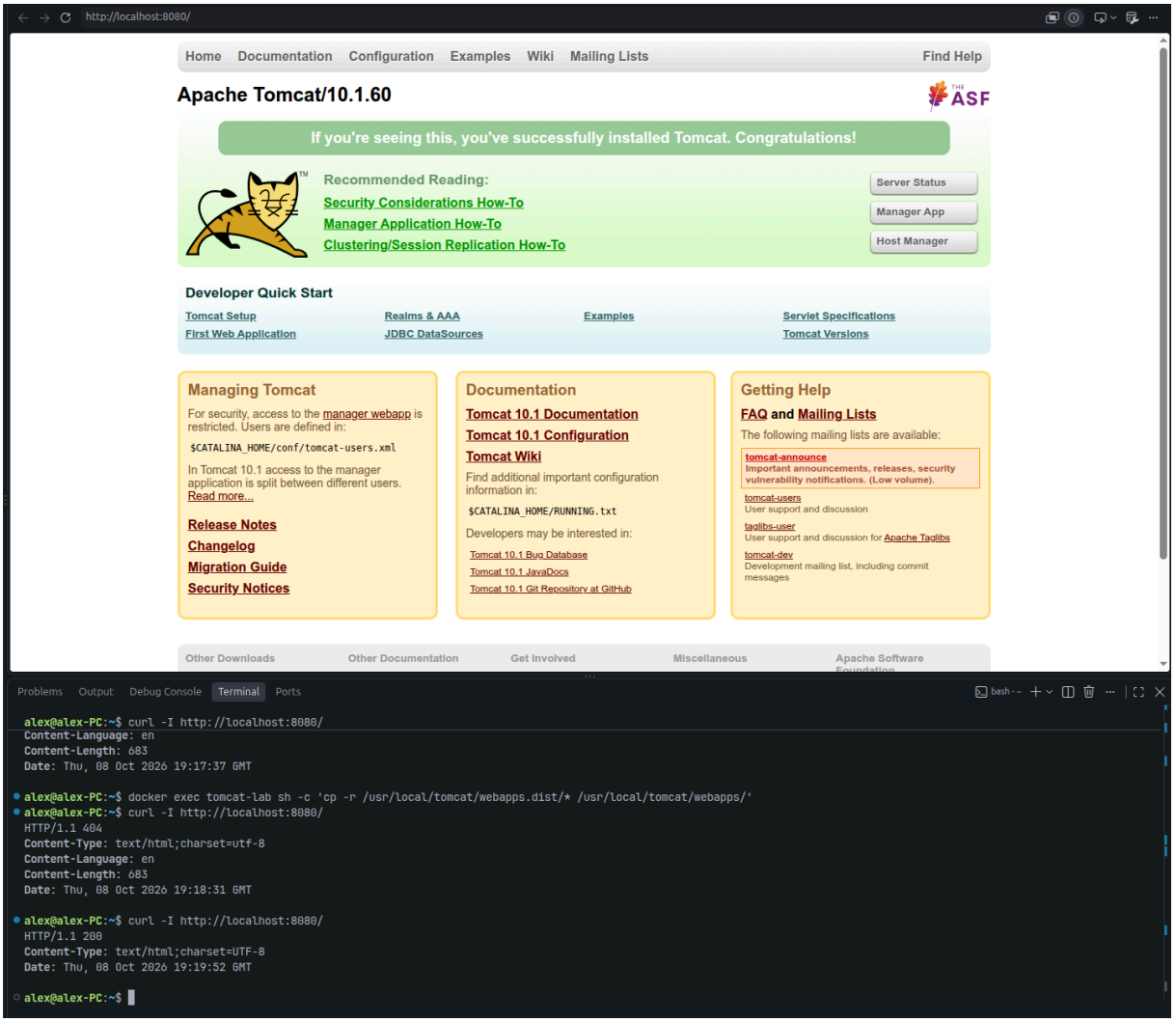
Obtendrás un 404, no un error de conexión: el servidor funciona, pero las imágenes oficiales vienen sin aplicaciones desplegadas. Restaura las que trae de serie y vuelve a comprobar:

```
docker exec tomcat-lab sh -c 'cp -r /usr/local/tomcat/webapps.dist/*
/usr/local/tomcat/webapps/'
curl -I http://localhost:8080/
```

Explica en la memoria por qué un 404 y un «conexión rehusada» apuntan a problemas completamente distintos.

Respuesta





El error `404 Not Found` y el error `Connection refused` se producen en capas diferentes del modelo de red:

- **Error `404 Not Found`:** se produce en la **capa de aplicación**. Indica que la conexión con el servidor se ha completado correctamente, pero el recurso solicitado no existe. Fue el error recibido tras levantar Tomcat, porque el directorio `webapps/` viene vacío por defecto y, por tanto, no hay ninguna página raíz que servir. Se corrigió copiando las aplicaciones que vienen de serie en `webapps.dist/`. De esta forma, al cargar `http://localhost:8080/` se muestra la página principal de Tomcat.
- **Error `Connection refused`:** se produce en la **capa de transporte**. El cliente intentó conectar con una IP y un puerto concretos, pero no había ningún proceso escuchando. En este ejemplo podría ocurrir porque Tomcat estuviera detenido, el contenedor no se hubiera levantado, el mapeo de puertos (`-p 8080:8080`) fuera incorrecto o un cortafuegos rechazara el tráfico. El paquete llega a la máquina, pero el sistema operativo rechaza la conexión al no haber nadie atendiendo ese puerto. Si el cortafuegos descartara los paquetes sin responder (`DROP` en lugar de `REJECT`), el cliente obtendría un *timeout* en lugar de un `Connection refused` .

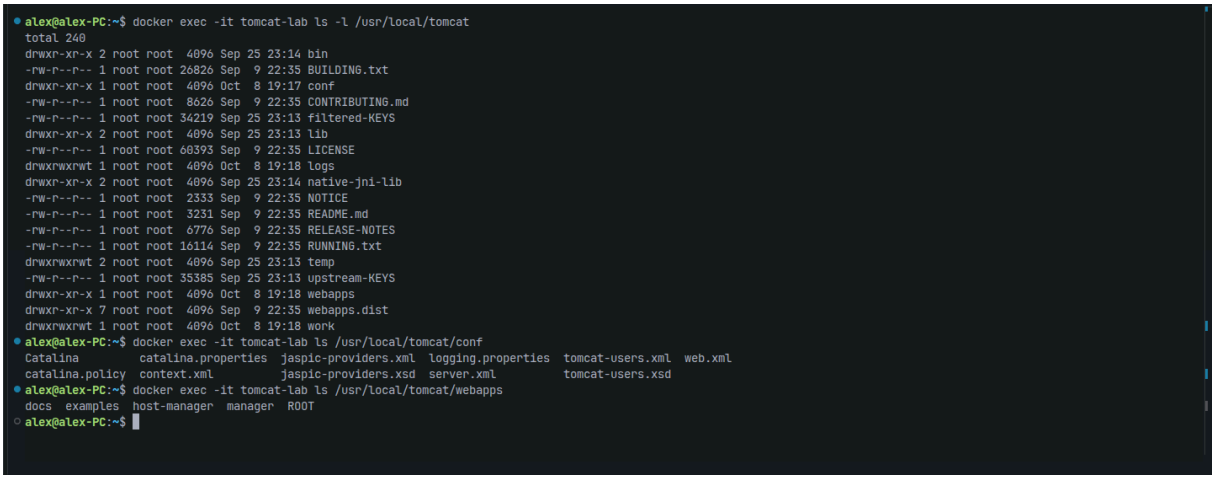
Paso 2. Recorrer la estructura

```
docker exec -it tomcat-lab ls -l /usr/local/tomcat
docker exec -it tomcat-lab ls /usr/local/tomcat/conf
docker exec -it tomcat-lab ls /usr/local/tomcat/webapps
```

Contrasta lo que ves con la tabla del apartado 11.3 y localiza en `conf/server.xml` el conector HTTP y el puerto de parada:

```
docker exec tomcat-lab grep -n 'Connector port' /usr/local/tomcat/conf/server.xml
docker exec tomcat-lab grep -n '<Server port' /usr/local/tomcat/conf/server.xml
```

Respuesta





Los directorios principales de Tomcat se encuentran en `/usr/local/tomcat`:

- **`bin/`**: scripts de control, como `startup.sh` y `shutdown.sh`.
- **`conf/`**: ficheros de configuración global. Destacan `server.xml`, para los parámetros de red y los conectores, y `tomcat-users.xml`, para el control de acceso y los roles.
- **`webapps/`**: directorio en el que se despliegan automáticamente las aplicaciones web, ya sea como directorios o como ficheros `.war`.
- **`lib/`**: librerías comunes compartidas por todas las aplicaciones.
- **`logs/`**: registros de actividad del servidor.
- **`temp/`** y **`work/`**: directorios de ficheros temporales y de almacenamiento de las JSP compiladas como servlets.

Dentro de `server.xml` se configuran los puertos y los conectores:

- **Conector HTTP (`<Connector port="8080" protocol="HTTP/1.1">`):** define el puerto `8080`, en el que

Tomcat recibe las peticiones de los clientes.

- **Puerto de parada (`<Server port="8005" shutdown="SHUTDOWN">`):** puerto de control interno. Al ejecutar una orden de apagado (como `shutdown.sh`), Tomcat envía la cadena `SHUTDOWN` a este puerto y realiza un **apagado controlado**: deja de aceptar nuevas conexiones, termina de procesar las peticiones pendientes, guarda el estado de las sesiones activas y libera los recursos. Solo escucha en `localhost` y puede desactivarse con `port="-1"` .

Paso 3. Construir una aplicación y empaquetarla como WAR

> *Un WAR no es más que un fichero comprimido con una estructura convenida. Créalo a mano:*
>
> ```
> mkdir -p ~/miapp/WEB-INF
> cd ~/miapp
> echo '<h1>Pagina estatica</h1>' > estatica.html
> ```
>
> *Añade una página generada en el servidor, para ver la diferencia con la anterior:*
>
> ```
> cat > index.jsp <<'EOF'
> <%@ page contentType="text/html; charset=UTF-8" %>
> <html><body>
>   <h1>Aplicacion desplegada en Tomcat</h1>
>   Generada el <%= new java.util.Date() %>
>   Servidor: <%= application.getServerInfo() %>
> </body></html>
> EOF
> zip -r ../miapp.war .      # también vale: jar -cvf ../miapp.war .
> ```

Respuesta

```
alex@alex-PC:~$ mkdir -p ~/miapp/WEB-INF
alex@alex-PC:~$ cd ~/miapp
alex@alex-PC:~/miapp$ echo '<h1>Pagina estatica</h1>' > estatica.html
alex@alex-PC:~/miapp$ cat > index.jsp <<'EOF'
> <%@ page contentType="text/html; charset=UTF-8" %>
<html><body>
<h1>Aplicacion desplegada en Tomcat</h1>
<p>Generada el <%= new java.util.Date() %></p>
<p>Servidor: <%= application.getServerInfo() %></p>
</body></html>
EOF
alex@alex-PC:~/miapp$ zip -r ../miapp.war .
  updating: index.jsp (deflated 24%)
  updating: WEB-INF/ (stored 0%)
  updating: estatica.html (stored 0%)
    adding: context.xml (deflated 42%)
    adding: tomcat-users.xml (deflated 60%)
alex@alex-PC:~/miapp$
```

El WAR se genera comprimiendo en ZIP el contenido de `~/miapp` : las páginas en la raíz y el directorio `WEB-INF/` , que es la estructura que Tomcat espera encontrar al desplegarlo.

Paso 4. Desplegar y observar el despliegue automático

```
docker cp ~/miapp.war tomcat-lab:/usr/local/tomcat/webapps/
docker logs --tail 20 tomcat-lab   # busca la línea de deployment
docker exec tomcat-lab ls /usr/local/tomcat/webapps
curl http://localhost:8080/miapp/
curl http://localhost:8080/miapp/estatica.html
```

Qué observar en este paso

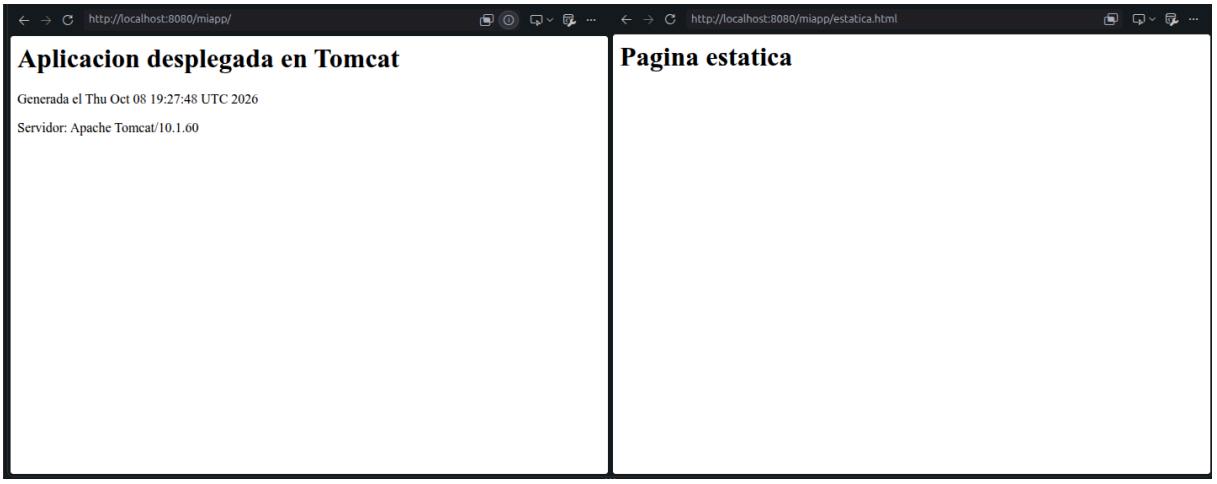
- *Tomcat ha descomprimido el WAR en un directorio con el mismo nombre: ese nombre es el que determina la ruta de acceso.*
- *Recarga `index.jsp` varias veces: la fecha cambia en cada petición. Haz lo mismo con `estatica.html` : el contenido es siempre idéntico. Ahí está, en una línea, la diferencia entre servidor web y servidor de aplicaciones.*
- *Mira en `work/` las clases que Tomcat ha generado al compilar la JSP la primera vez que se solicitó.*

Respuesta

```
alex@alex-PC:~$ docker cp ~/miapp.war tomcat-lab:/usr/local/tomcat/webapps/
Successfully copied 2.88kB (transferred 4.61kB) to tomcat-lab:/usr/local/tomcat/webapps/
alex@alex-PC:~$ docker logs --tail 20 tomcat-lab
08-Oct-2026 19:17:21.974 INFO [main] org.apache.catalina.core.AprLifecycleListener.lifecycleEvent Loaded Apache Tomcat Native library [2.0.16] using APR version [1.7.2].
08-Oct-2026 19:17:21.985 INFO [main] org.apache.catalina.core.AprLifecycleListener.initializeSSL OpenSSL successfully initialized [OpenSSL 3.0.13 30 Jan 2024]
08-Oct-2026 19:17:22.124 INFO [main] org.apache.coyote.AbstractProtocol.init Initializing ProtocolHandler ["http-nio-8080"]
08-Oct-2026 19:17:22.140 INFO [main] org.apache.catalina.startup.Catalina.load Server initialization in [309] milliseconds
08-Oct-2026 19:17:22.170 INFO [main] org.apache.catalina.core.StandardService.startInternal Starting service [Catalina]
08-Oct-2026 19:17:22.171 INFO [main] org.apache.catalina.core.StandardEngine.startInternal Starting Servlet engine: [Apache Tomcat/10.1.60]
08-Oct-2026 19:17:22.179 INFO [main] org.apache.coyote.AbstractProtocol.start Starting ProtocolHandler ["http-nio-8080"]
08-Oct-2026 19:17:22.187 INFO [main] org.apache.catalina.startup.Catalina.start Server startup in [46] milliseconds
08-Oct-2026 19:18:32.193 INFO [Catalina-utility-1] org.apache.catalina.startup.HostConfig.deployDirectory Deploying web application directory [/usr/local/tomcat/webapps/host-manager]
08-Oct-2026 19:18:32.367 INFO [Catalina-utility-1] org.apache.catalina.startup.HostConfig.deployDirectory Deployment of web application directory [/usr/local/tomcat/webapps/host-manager] has finished in [174] ms
08-Oct-2026 19:18:32.367 INFO [Catalina-utility-1] org.apache.catalina.startup.HostConfig.deployDirectory Deploying web application directory [/usr/local/tomcat/webapps/manager]
08-Oct-2026 19:18:32.382 INFO [Catalina-utility-1] org.apache.catalina.startup.HostConfig.deployDirectory Deployment of web application directory [/usr/local/tomcat/webapps/manager] has finished in [15] ms
08-Oct-2026 19:18:32.382 INFO [Catalina-utility-1] org.apache.catalina.startup.HostConfig.deployDirectory Deploying web application directory [/usr/local/tomcat/webapps/examples]
08-Oct-2026 19:18:32.488 INFO [Catalina-utility-1] org.apache.catalina.startup.HostConfig.deployDirectory Deployment of web application directory [/usr/local/tomcat/webapps/examples] has finished in [106] ms
08-Oct-2026 19:18:32.488 INFO [Catalina-utility-1] org.apache.catalina.startup.HostConfig.deployDirectory Deploying web application directory [/usr/local/tomcat/webapps/ROOT]
08-Oct-2026 19:18:32.499 INFO [Catalina-utility-1] org.apache.catalina.startup.HostConfig.deployDirectory Deployment of web application directory [/usr/local/tomcat/webapps/ROOT] has finished in [11] ms
08-Oct-2026 19:18:32.499 INFO [Catalina-utility-1] org.apache.catalina.startup.HostConfig.deployDirectory Deploying web application directory [/usr/local/tomcat/webapps/docs]
08-Oct-2026 19:18:32.507 INFO [Catalina-utility-1] org.apache.catalina.startup.HostConfig.deployDirectory Deployment of web application directory [/usr/local/tomcat/webapps/docs] has finished in [8] ms
08-Oct-2026 19:26:02.581 INFO [Catalina-utility-2] org.apache.catalina.startup.HostConfig.deployWAR Deploying web application archive [/usr/local/tomcat/webapps/miapp.war]
08-Oct-2026 19:26:02.597 INFO [Catalina-utility-2] org.apache.catalina.startup.HostConfig.deployWAR Deployment of web application archive [/usr/local/tomcat/webapps/miapp.war] has finished in [15] ms
alex@alex-PC:~$
```

```
alex@alex-PC:~$ docker exec tomcat-lab ls /usr/local/tomcat/webapps
docs
examples
host-manager
manager
miapp
miapp.war
ROOT
alex@alex-PC:~$ curl http://localhost:8080/miapp/

<html><body>
<h1>Aplicacion desplegada en Tomcat</h1>
<p>Generada el Thu Oct 08 19:27:20 UTC 2026</p>
<p>Servidor: Apache Tomcat/10.1.60</p>
</body></html>
alex@alex-PC:~$ curl http://localhost:8080/miapp/estatica.html
<h1>Pagina estatica</h1>
alex@alex-PC:~$
```




```
alex@alex-PC:~$ docker ps
CONTAINER ID   IMAGE              COMMAND             CREATED          STATUS          PORTS                                       NAMES
f7ab6213d0a4   tomcat:10.1-jdk17  "catalina.sh run"   15 minutes ago   Up 15 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   tomcat-lab
alex@alex-PC:~$ docker exec -it f7ab bash
root@f7ab6213d0a4:/usr/local/tomcat# cd /usr/local/tomcat/w
webapps/      webapps.dist/ work/
root@f7ab6213d0a4:/usr/local/tomcat# cd /usr/local/tomcat/work/Catalina/localhost/
root@f7ab6213d0a4:/usr/local/tomcat/work/Catalina/localhost# ls -la
total 32
drwxr-x--- 8 root root 4096 Oct  8 19:26 .
drwxr-x--- 3 root root 4096 Oct  8 19:18 ..
drwxr-x--- 2 root root 4096 Oct  8 19:18 docs
drwxr-x--- 2 root root 4096 Oct  8 19:18 examples
drwxr-x--- 2 root root 4096 Oct  8 19:18 host-manager
drwxr-x--- 2 root root 4096 Oct  8 19:18 manager
drwxr-x--- 3 root root 4096 Oct  8 19:27 miapp
drwxr-x--- 3 root root 4096 Oct  8 19:19 ROOT
root@f7ab6213d0a4:/usr/local/tomcat/work/Catalina/localhost# cd miapp/
root@f7ab6213d0a4:/usr/local/tomcat/work/Catalina/localhost/miapp# ls -la
total 12
drwxr-x--- 3 root root 4096 Oct  8 19:27 .
drwxr-x--- 8 root root 4096 Oct  8 19:26 ..
drwxr-x--- 3 root root 4096 Oct  8 19:27 org
root@f7ab6213d0a4:/usr/local/tomcat/work/Catalina/localhost/miapp# cd org/apache/jsp/
root@f7ab6213d0a4:/usr/local/tomcat/work/Catalina/localhost/miapp/org/apache/jsp# ls -la
total 24
drwxr-x--- 2 root root 4096 Oct  8 19:27 .
drwxr-x--- 3 root root 4096 Oct  8 19:27 ..
-rw-r----- 1 root root 6091 Oct  8 19:24 index_jsp.class
-rw-r----- 1 root root 5300 Oct  8 19:24 index_jsp.java
root@f7ab6213d0a4:/usr/local/tomcat/work/Catalina/localhost/miapp/org/apache/jsp#
```

Al copiar el fichero `.war` en el directorio `webapps/`, el **despliegue automático** de Tomcat (`autoDeploy`) lo detecta, lo descomprime en un directorio con el mismo nombre y despliega la aplicación (mensajes de *deployment* del log).

La ruta de acceso (*context path*) la determina el nombre del directorio, de forma que la URL base de la aplicación es, en este ejemplo, `http://localhost:8080/miapp/`. `index.jsp` es una **página dinámica** que el servidor de aplicaciones procesa en cada petición; como la fecha se obtiene con una expresión Java, muestra la hora actual cada vez. En cambio, `estatica.html` es una **página estática** y muestra siempre el mismo contenido.

Cuando se solicita la página JSP por primera vez, Tomcat la traduce a código fuente Java y la compila a **bytecode**. Estos ficheros se guardan en `work/` para evitar repetir la traducción en peticiones posteriores, lo que mejora el rendimiento.

Esto ocurre porque Tomcat no puede interpretar una página JSP tal cual. Su motor de JSP, **Jasper**, lee el fichero `.jsp` (que mezcla HTML y código Java), lo traduce a un **servlet** Java estándar en un fichero `.java` e inmediatamente lo compila a `.class` para que la máquina virtual de Java (**JVM**) pueda ejecutarlo. Cuando se solicita `index.jsp`, la JVM ejecuta el `.class` para generar el HTML final que se envía al cliente.

Contenido del `.java` generado:

```java
/*
 * Generated by the Jasper component of Apache Tomcat
 * Version: Apache Tomcat/10.1.60
 * Generated at: 2026-10-08 19:27:20 UTC
 * Note: The last modified time of this file was set to
 *       the last modified time of the source file after
 *       generation to assist with modification tracking.
 */
package org.apache.jsp;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.jsp.*;

public final class index_jsp extends org.apache.jasper.runtime.HttpJspBase
    implements org.apache.jasper.runtime.JspSourceDependent,
                 org.apache.jasper.runtime.JspSourceImports,
                 org.apache.jasper.runtime.JspSourceDirectives {

  private static final jakarta.servlet.jsp.JspFactory _jspxFactory =
          jakarta.servlet.jsp.JspFactory.getDefaultFactory();

  private static java.util.Map<java.lang.String,java.lang.Long> _jspx_dependants;

  private static final java.util.Set<java.lang.String> _jspx_imports_packages;

  private static final java.util.Set<java.lang.String> _jspx_imports_classes;

  static {
    _jspx_imports_packages = new java.util.LinkedHashSet<>(4);
    _jspx_imports_packages.add("jakarta.servlet");
    _jspx_imports_packages.add("jakarta.servlet.http");
    _jspx_imports_packages.add("jakarta.servlet.jsp");
    _jspx_imports_classes = null;
  }

  private volatile jakarta.el.ExpressionFactory _el_expressionfactory;
  private volatile org.apache.tomcat.InstanceManager _jsp_instancemanager;

  public java.util.Map<java.lang.String,java.lang.Long> getDependants() {
    return _jspx_dependants;
  }
```

```
  public java.util.Set<java.lang.String> getPackageImports() {
    return _jspx_imports_packages;
  }

  public java.util.Set<java.lang.String> getClassImports() {
    return _jspx_imports_classes;
  }

  public boolean getErrorOnELNotFound() {
    return false;
  }

  public jakarta.el.ExpressionFactory _jsp_getExpressionFactory() {
    if (_el_expressionfactory == null) {
      synchronized (this) {
        if (_el_expressionfactory == null) {
          _el_expressionfactory =
_jspxFactory.getJspApplicationContext(getServletConfig().getServletContext()).getExpressionFact
ory();
        }
      }
    }
    return _el_expressionfactory;
  }

  public org.apache.tomcat.InstanceManager _jsp_getInstanceManager() {
    if (_jsp_instancemanager == null) {
      synchronized (this) {
        if (_jsp_instancemanager == null) {
          _jsp_instancemanager =
org.apache.jasper.runtime.InstanceManagerFactory.getInstanceManager(getServletConfig());
        }
      }
    }
    return _jsp_instancemanager;
  }

  public void _jspInit() {
  }

  public void _jspDestroy() {
  }
```

```
  public void _jspService(final jakarta.servlet.http.HttpServletRequest request, final
jakarta.servlet.http.HttpServletResponse response)
      throws java.io.IOException, jakarta.servlet.ServletException {

    if (!jakarta.servlet.DispatcherType.ERROR.equals(request.getDispatcherType())) {
      final java.lang.String _jspx_method = request.getMethod();
      if ("OPTIONS".equals(_jspx_method)) {
        response.setHeader("Allow","GET, HEAD, POST, OPTIONS");
        return;
      }
      if (!"GET".equals(_jspx_method) && !"POST".equals(_jspx_method) &&
!"HEAD".equals(_jspx_method)) {
        response.setHeader("Allow","GET, HEAD, POST, OPTIONS");
        response.sendError(HttpServletResponse.SC_METHOD_NOT_ALLOWED, "JSPs only permit GET,
POST or HEAD. Jasper also permits OPTIONS");
        return;
      }
    }

    final jakarta.servlet.jsp.PageContext pageContext;
    jakarta.servlet.http.HttpSession session = null;
    final jakarta.servlet.ServletContext application;
    final jakarta.servlet.ServletConfig config;
    jakarta.servlet.jsp.JspWriter out = null;
    final java.lang.Object page = this;
    jakarta.servlet.jsp.JspWriter _jspx_out = null;
    jakarta.servlet.jsp.PageContext _jspx_page_context = null;


    try {
      response.setContentType("text/html; charset=UTF-8");
      pageContext = _jspxFactory.getPageContext(this, request, response,
                  null, true, 8192, true);
      _jspx_page_context = pageContext;
      application = pageContext.getServletContext();
      config = pageContext.getServletConfig();
      session = pageContext.getSession();
      out = pageContext.getOut();
      _jspx_out = out;

      out.write("\n");
      out.write("<html><body>\n");
      out.write("<h1>Aplicacion desplegada en Tomcat</h1>\n");
```

```
      out.write("<p>Generada el ");
      out.print( new java.util.Date() );
      out.write("</p>\n");
      out.write("<p>Servidor: ");
      out.print( application.getServerInfo() );
      out.write("</p>\n");
      out.write("</body></html>\n");
    } catch (java.lang.Throwable t) {
      if (!(t instanceof jakarta.servlet.jsp.SkipPageException)){
        out = _jspx_out;
        if (out != null && out.getBufferSize() != 0)
          try {
            if (response.isCommitted()) {
              out.flush();
            } else {
              out.clearBuffer();
            }
          } catch (java.io.IOException e) {}
        if (_jspx_page_context != null) _jspx_page_context.handlePageException(t);
        else throw new ServletException(t);
      }
    } finally {
      _jspxFactory.releasePageContext(_jspx_page_context);
    }
  }
}
```

> **Nota:** *el fichero `.war` no se debe eliminar, porque el despliegue automático de Tomcat replegaría la aplicación y borraría el directorio descomprimido.*

Paso 5. La consola de administración

> *Habilita un usuario con permisos de gestión editando `conf/tomcat-users.xml` :*
>
> ```
> <role rolename="manager-gui"/>
> <user username="admin" password="1234" roles="manager-gui"/>
> ```
>
> *Por defecto la aplicación de gestión solo admite conexiones desde la propia máquina, así que desde un contenedor hay que permitir el acceso externo en `webapps/manager/META-INF/context.xml` . Documenta qué válvula lo restringe y por qué esa restricción existe. Después reinicia el contenedor, accede a http://localhost:8080/manager/html y comprueba que puedes ver, parar y volver a arrancar tu aplicación desde la consola.*

Respuesta

```
alex@alex-PC:~$ docker ps
CONTAINER ID   IMAGE              COMMAND             CREATED          STATUS          PORTS                                       NAMES
f7ab6213d0a4   tomcat:10.1-jdk17  "catalina.sh run"   27 minutes ago   Up 27 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp  tomcat-lab
alex@alex-PC:~$ docker exec -it f7ab bash
root@f7ab6213d0a4:/usr/local/tomcat# ls
bin           conf            filtered-KEYS  LICENSE  native-jni-lib  README.md      RUNNING.txt  upstream-KEYS  webapps.dist
BUILDING.txt  CONTRIBUTING.md  lib            logs     NOTICE          RELEASE-NOTES  temp         webapps        work
root@f7ab6213d0a4:/usr/local/tomcat# cd conf
root@f7ab6213d0a4:/usr/local/tomcat/conf# ls -la
total 244
drwxr-xr-x 1 root root   4096 Oct  8 19:17 .
drwxr-xr-x 1 root root   4096 Sep 25 23:14 ..
drwxr-x--- 3 root root   4096 Oct  8 19:17 Catalina
-rw-r--r-- 1 root root  12926 Sep  9 22:35 catalina.policy
-rw-r--r-- 1 root root   7717 Sep  9 22:35 catalina.properties
-rw-r--r-- 1 root root   1411 Sep  9 22:35 context.xml
-rw-r--r-- 1 root root   1149 Sep  9 22:35 jaspic-providers.xml
-rw-r--r-- 1 root root   2313 Sep  9 22:35 jaspic-providers.xsd
-rw-r--r-- 1 root root   4003 Sep  9 22:35 logging.properties
-rw-r--r-- 1 root root   7126 Sep  9 22:35 server.xml
-rw-r--r-- 1 root root   2756 Sep  9 22:35 tomcat-users.xml
-rw-r--r-- 1 root root   2558 Sep  9 22:35 tomcat-users.xsd
-rw-r--r-- 1 root root 173862 Sep  9 22:35 web.xml
root@f7ab6213d0a4:/usr/local/tomcat/conf#
```

```
alex@alex-PC:~$ docker cp tomcat-lab:/usr/local/tomcat/conf/tomcat-users.xml .
Successfully copied 2.76kB (transferred 4.61kB) to /home/alex/.
alex@alex-PC:~$ nano tomcat-users.xml
```

```
 GNU nano 8.7.1                                                tomcat-users.xml *

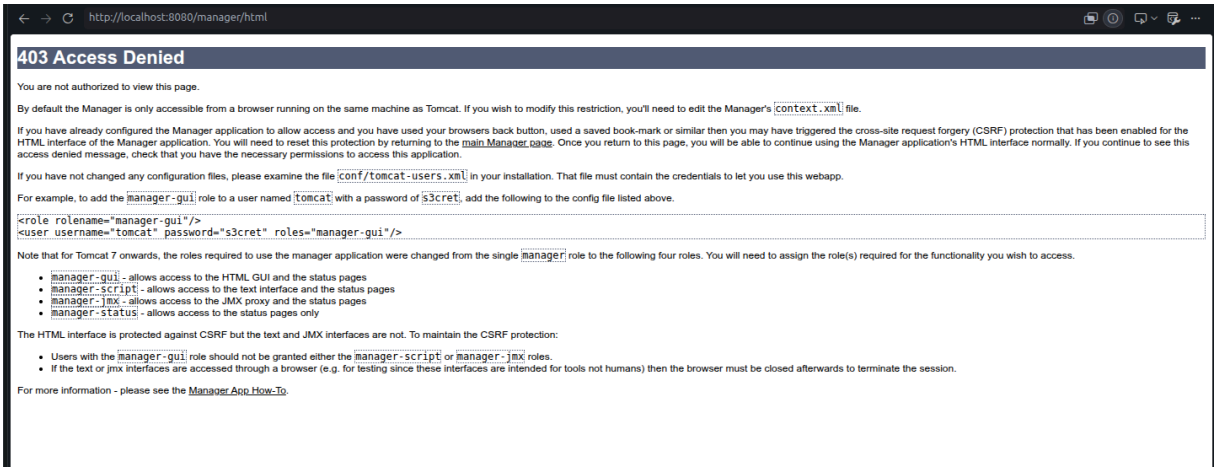
  The users below are wrapped in a comment and are therefore ignored. If you
  wish to configure one or more of these users for use with the manager web
  application, do not forget to remove the <!.. ..> that surrounds them. You
  will also need to set the passwords to something appropriate.
-->
<!--
  <user username="admin" password="<must-be-changed>" roles="manager-gui"/>
  <user username="robot" password="<must-be-changed>" roles="manager-script"/>
-->
<!--
  The sample user and role entries below are intended for use with the
  examples web application. They are wrapped in a comment and thus are ignored
  when reading this file. If you wish to configure these users for use with the
  examples web application, do not forget to remove the <!.. ..> that surrounds
  them. You will also need to set the passwords to something appropriate.
-->
<!--
  <role rolename="tomcat"/>
  <role rolename="role1"/>
  <user username="tomcat" password="<must-be-changed>" roles="tomcat"/>
  <user username="both" password="<must-be-changed>" roles="tomcat,role1"/>
  <user username="role1" password="<must-be-changed>" roles="role1"/>
-->

  <role rolename="manager-gui"/>
  <user username="admin" password="1234" roles="manager-gui"/>

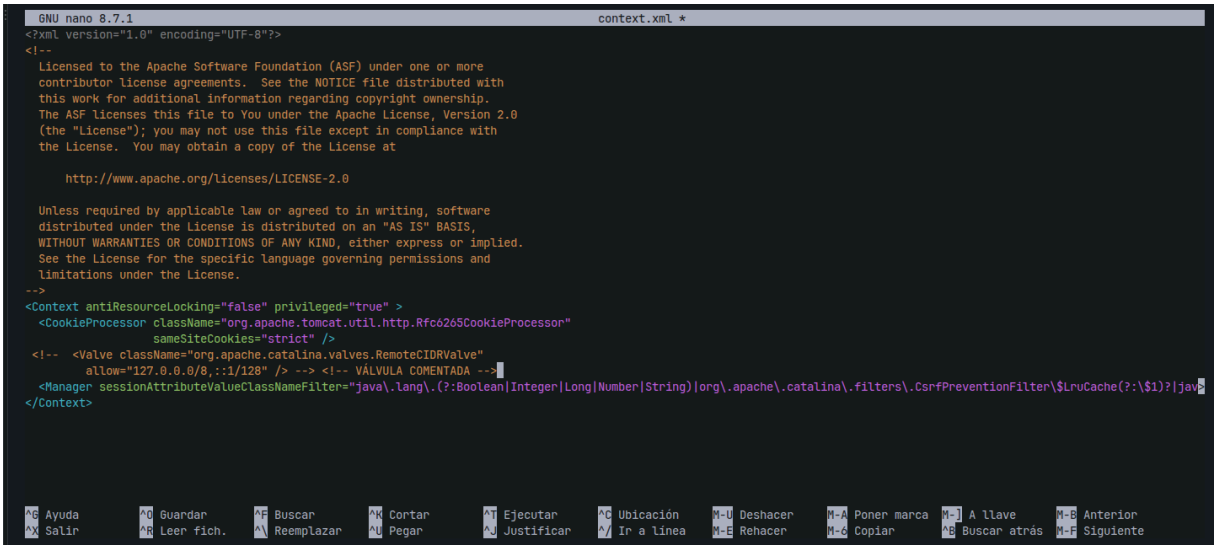
</tomcat-users>

^G Ayuda      ^O Guardar     ^F Buscar      ^K Cortar      ^T Ejecutar    ^C Ubicación   M-U Deshacer   M-A Poner marca  M-] A llave     M-B Anterior
^X Salir      ^R Leer fich.  ^\ Reemplazar  ^U Pegar      ^J Justificar  ^/ Ir a línea  M-E Rehacer    M-6 Copiar       ^B Buscar atrás M-F Siguiente
```

```
alex@alex-PC:~$ docker cp tomcat-lab:/usr/local/tomcat/conf/tomcat-users.xml .
Successfully copied 2.76kB (transferred 4.61kB) to /home/alex/.
alex@alex-PC:~$ nano tomcat-users.xml
alex@alex-PC:~$ docker cp tomcat-users.xml tomcat-lab:/usr/local/tomcat/conf/
Successfully copied 2.85kB (transferred 4.61kB) to tomcat-lab:/usr/local/tomcat/conf/
alex@alex-PC:~$
```

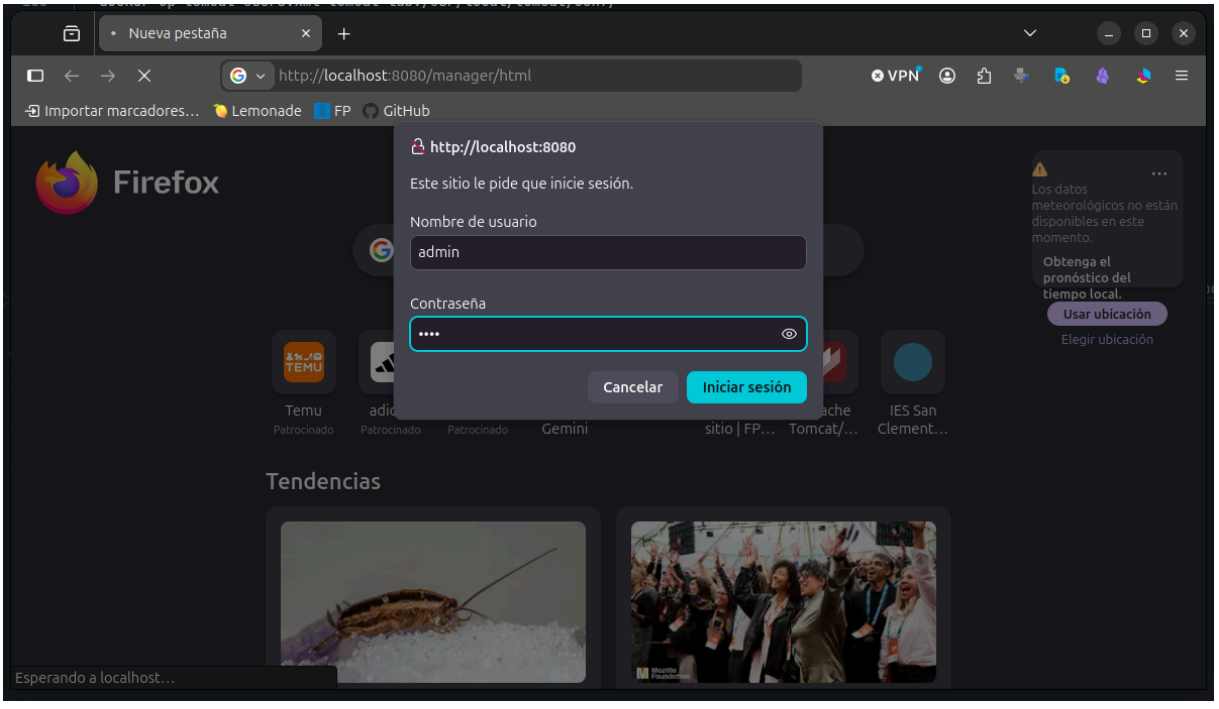


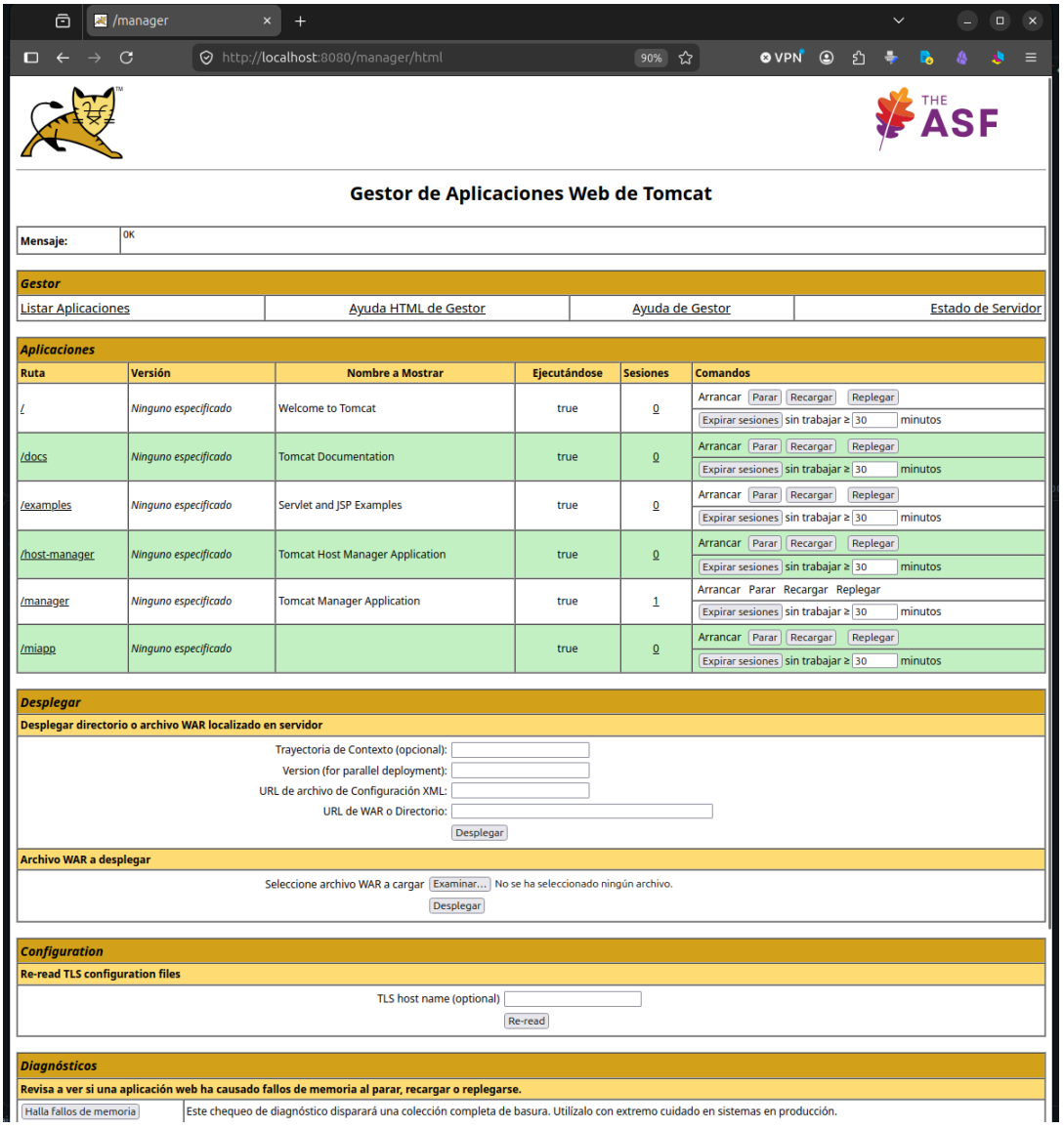

```
alex@alex-PC:~$ docker cp tomcat-lab:/usr/local/tomcat/webapps/manager/META-INF/context.xml .
Successfully copied 1.33kB (transferred 3.07kB) to /home/alex/.
alex@alex-PC:~$ nano context.xml
```


GNU nano 8.7.1 context.xml *
<?xml version="1.0" encoding="UTF-8"?>
<!--
Licensed to the Apache Software Foundation (ASF) under one or more
contributor license agreements. See the NOTICE file distributed with
this work for additional information regarding copyright ownership.
The ASF licenses this file to You under the Apache License, Version 2.0
(the "License"); you may not use this file except in compliance with
the License. You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
-->
<Context antiResourceLocking="false" privileged="true" >
<CookieProcessor className="org.apache.tomcat.util.http.Rfc6265CookieProcessor"
sameSiteCookies="strict" />
<!-- <Valve className="org.apache.catalina.valves.RemoteCIDRValve"
allow="127.0.0.0/8,::1/128" /> --> <!-- VÁLVULA COMENTADA -->
<Manager sessionAttributeValueClassNameFilter="java\.lang\.(?:Boolean|Integer|Long|Number|String)|org\.apache\.catalina\.filters\.CsrfPreventionFilter\$LruCache(?:\$1)?|jav
</Context>
^G Ayuda ^O Guardar ^F Buscar ^K Cortar ^T Ejecutar ^C Ubicación M-U Deshacer M-A Poner marca M-] A llave M-B Anterior
^X Salir ^R Leer fich. ^\ Reemplazar ^U Pegar ^J Justificar ^/ Ir a línea M-E Rehacer M-6 Copiar ^B Buscar atrás M-F Siguiente


alex@alex-PC:~$ docker cp tomcat-lab:/usr/local/tomcat/webapps/manager/META-INF/context.xml .
Successfully copied 1.33kB (transferred 3.07kB) to /home/alex/.
alex@alex-PC:~$ nano context.xml
alex@alex-PC:~$ docker cp context.xml tomcat-lab:/usr/local/tomcat/webapps/manager/META-INF/context.xml
Successfully copied 1.37kB (transferred 3.07kB) to tomcat-lab:/usr/local/tomcat/webapps/manager/META-INF/context.xml
alex@alex-PC:~$

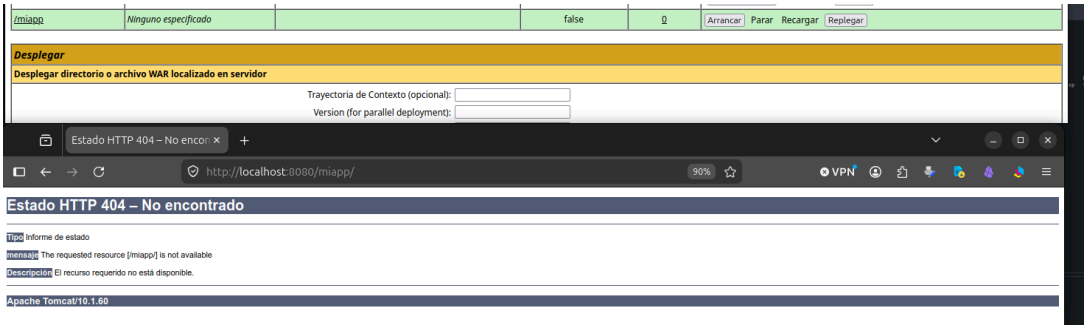

alex@alex-PC:~$ docker cp tomcat-lab:/usr/local/tomcat/webapps/manager/META-INF/context.xml .
Successfully copied 1.33kB (transferred 3.07kB) to /home/alex/.
alex@alex-PC:~$ nano context.xml
alex@alex-PC:~$ docker cp context.xml tomcat-lab:/usr/local/tomcat/webapps/manager/META-INF/context.xml
Successfully copied 1.37kB (transferred 3.07kB) to tomcat-lab:/usr/local/tomcat/webapps/manager/META-INF/context.xml
alex@alex-PC:~$ docker restart tomcat-lab
tomcat-lab
alex@alex-PC:~$ ^C
alex@alex-PC:~$


Nueva pestaña
http://localhost:8080/manager/html
VPN
Importar marcadores...
Lemonade
FP
GitHub
Firefox
http://localhost:8080
Este sitio le pide que inicie sesión.
Nombre de usuario
admin
Contraseña
Cancelar
Iniciar sesión
Los datos meteorológicos no están disponibles en este momento.
Obtenga el pronóstico del tiempo local.
Usar ubicación
Elegir ubicación
Temu
Patrocinado
Gemini
IES San Clement...
Tendencias
Esperando a localhost...



La restricción la aplica la válvula `RemoteCIDRValve` del fichero `webapps/manager/META-INF/context.xml`, que solo admite peticiones desde `127.0.0.0/8` y `::1/128` (la propia máquina). Como las peticiones al contenedor llegan desde la red de Docker (`172.17.0.1`), se comentó la válvula para permitir el acceso. Esta restricción existe porque el **Manager** permite desplegar y ejecutar código en el servidor y, si las credenciales no son robustas, cualquiera podría hacerlo desde la red.

En esta imagen se observa cómo es posible detener la aplicación `miapp` desde la consola de administración, así como volver a arrancarla o replegarla:

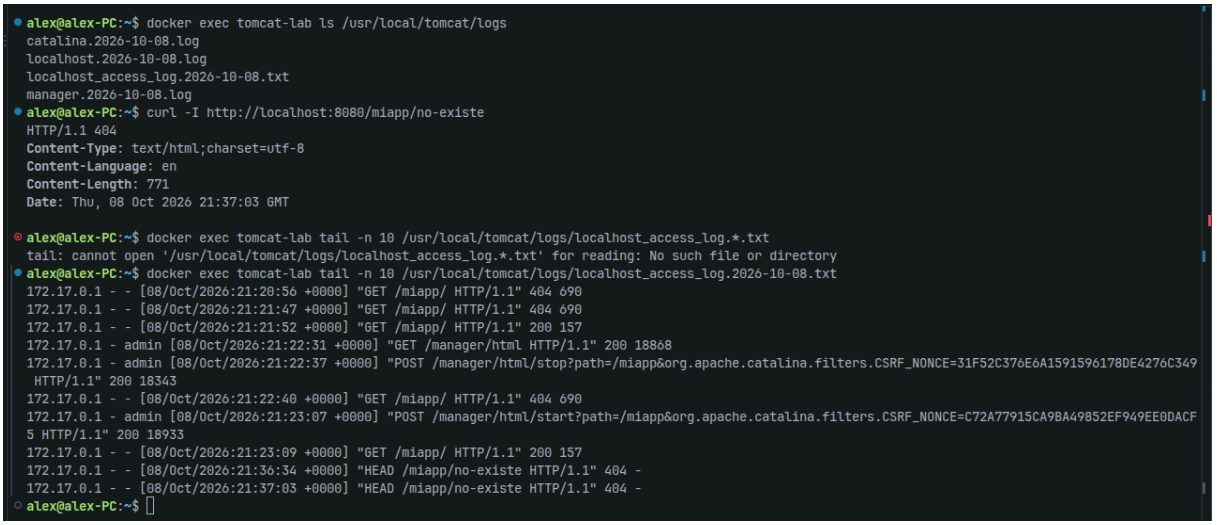


Paso 6. Registros y diagnóstico

```
docker exec tomcat-lab ls /usr/local/tomcat/logs
curl -I http://localhost:8080/miapp/no-existe
docker exec tomcat-lab tail -n 10 /usr/local/tomcat/logs/localhost_access_log.*.txt
```

Identifica tu petición fallida en el registro de acceso y explica cada campo de la línea.

Respuesta



Dentro de `logs/` hay varios ficheros de registro de la actividad del servidor. En `localhost_access_log.*.txt` se registran las peticiones HTTP recibidas. Al pedir un recurso que no existe se genera un error `404 Not Found`, que queda reflejado en el registro de acceso así:

```
172.17.0.1 - - [08/Oct/2026:21:37:03 +0000] "HEAD /miapp/no-existe HTTP/1.1" 404 -
```

- `172.17.0.1`**:** dirección IP del cliente que realiza la petición.
- `-`**:** identificador remoto del cliente. No suele usarse, así que aparece `-`.
- `-`**:** usuario autenticado, si la página requiere credenciales. Si el acceso es anónimo, se muestra `-`.
- `[08/Oct/2026:21:37:03 +0000]`**:** fecha, hora y zona horaria en que el servidor recibió la petición.
- `"HEAD /miapp/no-existe HTTP/1.1"`**: línea de petición** enviada por el cliente, formada por el método HTTP (`HEAD`), la ruta solicitada (`/miapp/no-existe`) y la versión del protocolo.
- `404`**:** código de estado HTTP devuelto por el servidor.
- `-`**:** tamaño en bytes del cuerpo de la respuesta. Aparece `-` (0 bytes) porque una respuesta a `HEAD` no lleva cuerpo.

Paso 7. Replegar la aplicación

```
docker exec tomcat-lab rm /usr/local/tomcat/webapps/miapp.war
sleep 5 && docker exec tomcat-lab ls /usr/local/tomcat/webapps
curl -I http://localhost:8080/miapp/
```

Comprueba que al eliminar el WAR desaparece también el directorio desplegado. Explica qué implicaciones tiene este comportamiento en un despliegue real.

Respuesta

Al borrar el fichero `.war` se comprueba que también desaparece el directorio de la aplicación. Es necesario esperar (de ahí el `sleep 5`) antes de listar `webapps/`, porque Tomcat tiene un hilo que revisa ese directorio en segundo plano y, cuando detecta que se ha eliminado el `.war`, detiene la aplicación, libera la memoria y, finalmente, borra su directorio.

A continuación se hace una petición `HEAD` a la aplicación. Como la ruta ya no existe, devuelve un `404 Not Found`.

En un entorno de producción, este comportamiento tiene la **ventaja** de que una aplicación se actualiza fácilmente: los cambios hechos al borrar o reemplazar el `.war` se aplican automáticamente sin reiniciar el servidor. Sin embargo, también tiene un **inconveniente**: se pierden los datos guardados dentro del directorio de la aplicación, como los ficheros subidos por los usuarios. Para evitarlo, los datos persistentes deben almacenarse fuera de la estructura del `.war`, en volúmenes o bases de datos externos.

Paso 8. Requisitos de implantación

A partir de todo lo anterior, redacta la tabla de requisitos previos que habría exigido desplegar esta aplicación sobre una máquina real y no sobre un contenedor, siguiendo las categorías del apartado 10.1: sistema operativo, dependencias, recursos, red y puertos, seguridad y observabilidad.

Respuesta

Requisito	Qué exigiría esta aplicación en una máquina real
Sistema operativo	Linux de 64 bits, recomendado para servidores (Ubuntu Server 24.04 LTS o Debian 12). Tomcat también funciona en Windows y macOS. Si los repositorios de la distribución no ofrecen Tomcat 10.1, se instala desde el paquete oficial de Apache.
Dependencias de software	Java 11 o superior (JDK o JRE), el mínimo de Tomcat 10.1. Para igualar el contenedor (`tomcat:10.1-jdk17`) se instala OpenJDK 17 y se define `JAVA_HOME`. No hace falta cliente de base de datos, porque la aplicación no usa ninguna.
Recursos de hardware	1 núcleo y 2 GB de RAM para desarrollo; 2 núcleos y 4 GB para producción. Al menos 1 GB de disco libre para Java, Tomcat, el `.war` y los directorios `logs/` y `work/`.

Requisito	Qué exigiría esta aplicación en una máquina real
Red y puertos	IP fija, nombre de dominio y registro DNS. Abrir en el cortafuegos el `8080` (HTTP) y, si se configura TLS, el `8443` (HTTPS), o bien el `80` y el `443` de un proxy inverso delante de Tomcat. El puerto de parada `8005` solo escucha en `localhost` y no se abre al exterior.
Seguridad	Ejecutar Tomcat con un usuario propio sin privilegios de `root`. Quitar las aplicaciones de ejemplo, limitar el **Manager** por IP con contraseñas robustas, servir por HTTPS y mantener actualizados el sistema, Java y Tomcat. Hacer copias de seguridad de `conf/` y del `.war`.
Observabilidad	Revisar los registros de `logs/` y configurar su rotación con `logrotate` para que no llenen el disco.

Cuestiones

1. Enumera cinco servicios que proporciona un servidor de aplicaciones y que un servidor web no ofrece. Señala cuáles has podido observar en la práctica.

1. **Entorno de ejecución:** ofrece una plataforma capaz de ejecutar programas y realizar operaciones complejas que un servidor web no permite. Así puede generar contenido dinámico, actualizado en tiempo real y personalizado para cada usuario.
2. **Conexión a bases de datos:** gestiona las conexiones para operar con bases de datos de forma segura. Las operaciones críticas se manejan mediante **transacciones**, que se completan solo si todos los pasos son correctos y se revierten en caso contrario (por ejemplo, una compra en línea).
3. **Gestión de sesiones:** conserva los datos de sesión del usuario durante su interacción con el sitio.
4. **Seguridad y autenticación:** administra los permisos de acceso, la verificación de identidad, el cifrado, el control del despliegue de la aplicación, etc.
5. **Servicios de comunicación:** a diferencia de los servidores web, que trabajan fundamentalmente con HTTP, permiten intercambiar datos mediante otros protocolos y tecnologías, como gRPC o servicios REST con JSON.

En esta práctica se pudieron observar:

- **Punto 1:** `index.jsp` genera la fecha en cada petición, y Tomcat la traduce a un servlet y la compila a `.class` en `work/` para ejecutarla en la JVM (paso 4).
- **Punto 4:** el acceso al **Manager** exige un usuario con el rol `manager-gui` definido en `tomcat-users.xml` y queda restringido por IP con la válvula `RemoteCIDRValve` (respuesta `403`). Desde la consola se controla el despliegue de la aplicación: pararla, arrancarla y replegarla (paso 5).

---

2. ¿Qué contiene realmente un fichero WAR? Descríbelo a partir del que has construido.

Un fichero `.war` (*Web Application Archive*) es un fichero comprimido estándar, en formato ZIP, que empaqueta una aplicación web Java con una estructura de directorios estandarizada, observable en el artefacto desplegado:

- **Raíz:** las páginas y recursos accesibles públicamente.
- **Directorio protegido** `WEB-INF/` **:** no es accesible desde el navegador y contiene:
  - `web.xml` **:** descriptor de despliegue con la configuración y el mapeo de la aplicación.
  - `classes/` **:** clases compiladas de la aplicación y JavaBeans.
  - `lib/` **:** librerías externas y dependencias necesarias para la ejecución.

El WAR construido solo contiene `index.jsp` y `estatica.html` en la raíz y un `WEB-INF/` vacío: `web.xml` es opcional desde Servlet 3.0 y la JSP la compila Tomcat al desplegarla.

---

3. ¿Por qué `index.jsp` cambia en cada recarga y `estatica.html` no?

Porque `estatica.html` es un fichero **HTML estático**: no cambia y se lee y transmite tal cual al cliente que lo solicita. En cambio, `index.jsp` es una **página dinámica** basada en JSP, cuyo código procesa el motor de Tomcat en el servidor. En este caso, el código incluye una llamada que imprime la fecha y hora actuales. Como ese contenido se genera en tiempo de ejecución, el resultado que se envía al cliente varía cada vez que se solicita la página.

---

4. ¿Qué ventaja tiene el despliegue automático por copia del artefacto y qué riesgo introduce en producción?

La **ventaja** es que facilita el **despliegue continuo** (*continuous deployment*): la aplicación se actualiza en caliente y de forma rápida reemplazando el fichero `.war`, sin reiniciar el servidor.

También supone **riesgos**: si al reemplazar el fichero se produce un error o se borra el `.war`, la aplicación se repliega de inmediato. Además, se pierde cualquier información almacenada dentro del directorio de la aplicación.

---

5. Si esta aplicación recibiera mucho tráfico de imágenes y hojas de estilo, ¿cómo reorganizarías el despliegue? Relaciónalo con el apartado 11.4.

Lo adecuado sería situar un servidor web, como Nginx o Apache, delante del servidor de aplicaciones, actuando como **proxy inverso**. Esto aporta varias ventajas:

- **Optimización de recursos estáticos:** el servidor web es muy eficiente sirviendo contenido estático y libera a Tomcat de esa tarea, de modo que este se encarga solo de la lógica dinámica.
- **Seguridad:** Tomcat no queda expuesto directamente a la red, lo que lo protege y oculta la estructura interna del servidor de aplicaciones.
- **Escalabilidad:** el proxy permite balancear la carga entre varias instancias de Tomcat en momentos de mucho tráfico.
